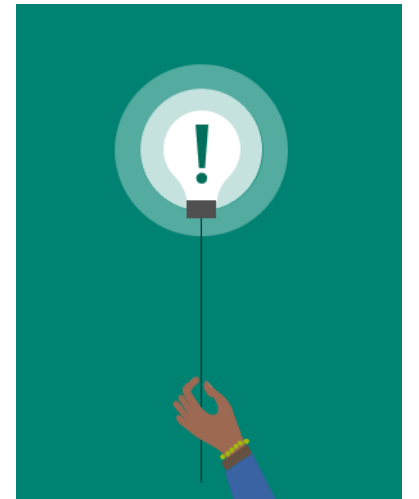


Mastering parameters of advanced functions

Objectives

After completing this learning, you will be able to:

- Understand the Parameter attribute
- Understand how to extend special properties for parameters
- Understand parameter validation attributes
- Understand how to use parameter validation
- Understand risk mitigation parameters
- Understand how to implement risk mitigation
- Understand comment-based help
- Understand how to implement the `OutputType()` attribute



Parameter Attribute

- Parameter attribute defines special properties:
 - Mandatory (boolean)
 - Position (integer)
 - ParameterSetName (string)
 - ValueFromPipeline (boolean)
 - ValueFromPipelineByPropertyName (boolean)
 - ValueFromRemainingArguments (boolean)
 - HelpMessage (string)

```
Param(  
    [parameter(Mandatory=$true, ValueFromPipeline=$true,  
                HelpMessage='The username to make edits to',  
                ValueFromPipelineByPropertyName=$true)]  
    [string] $username
```

Parameter attribute syntax

Syntax – Single Argument

```
function <name> {  
    Param (  
        [parameter(Argument=value)]  
        $ParameterName  
    )  
}
```

Parentheses that enclose the argument and its value must follow "Parameter" keyword with no intervening space

Syntax – Multiple Arguments

```
function <name> {  
    Param (  
        [parameter(Argument1=value1,  
                    Argument2=value2)]  
        $ParameterName  
    )  
}
```

Commas separate arguments

ParameterSetName



```
Param (  
    [parameter(ParameterSetName="Machine")]  
    [String[]] $MachineName,  
    [parameter(ParameterSetName="User")]  
    [String[]] $UserName  
)
```

Specifies parameter set to which a parameter belongs.

If no parameter set is specified, parameter belongs to all the parameter sets defined by the function
To be unique, each parameter set must have at least one parameter that is not a member of any other parameter set.

Mandatory and position



Mandatory

```
Param ([parameter(Mandatory=$true)] [String[]] $MachineName)
```

Position

```
Param ([parameter(Position=0)] [String[]] $MachineName)
```

- By default, all function parameters are positional
- PowerShell assigns position numbers to parameters in the order in which the parameters are declared in the function
- To disable, set PositionalBinding argument of CmdletBinding attribute to \$False
- Position argument takes precedence over PositionalBinding argument for the parameters on which it is declared

HelpMessage and Remaining Arguments



HelpMessage

```
Param (
    [parameter(Mandatory=$true,
                HelpMessage="Enter computer names separated by commas.")]
    [String[]] $ComputerName
)
```

Displays a message when a mandatory parameter value is missing

ValueFromRemainingArguments

```
Param(
    [parameter(ValueFromRemainingArguments=$true)]
    [String[]] $ComputerName
)
```

Parameter accepts all values that are not already assigned

Value From Remaining Arguments

Without NoValueFromRemainingArguments

```
function NoValueFromRemainingArguments {  
    Param($Surname,$GivenName,$MiddleOrOther)  
  
    Write-Host "`$Surname $Surname"  
    Write-Host "`$GivenName $GivenName"  
    Write-Host "`$MiddleOrOther $MiddleOrOther"  
}
```

```
PS C:\> NoValueFromRemainingArguments Jane Doe Sally Smith  
$Surname Jane  
$GivenName Doe  
$MiddleOrOther Sally
```

Value From Remaining Arguments cont.

With ValueFromRemainingArguments

```
function ValueFromRemainingArguments {  
    Param([Parameter(Mandatory=$true)]$Surname,  
          [Parameter(Mandatory=$true)]$GivenName,  
          [Parameter(Mandatory=$false,  
            ValueFromRemainingArguments=$true)]$MiddleOrOther  
        )  
    Write-Host "`$Surname $Surname"  
    Write-Host "`$GivenName $GivenName"  
    Write-Host "`$MiddleOrOther $MiddleOrOther"  
}
```

```
PS C:\> ValueFromRemainingArguments Jane Doe Sally Smith  
$Surname Jane  
$GivenName Doe  
$MiddleOrOther Sally Smith
```

Demonstration

Parameter Attributes



Questions?



Parameter Validation Attributes

- Validation attributes provide built-in validation capabilities to minimize script logic and allow the PowerShell engine to handle it instead
- Runs check before any code is executed
- Developer may decide to accept 'garbage in' and deal with inconsistencies of data in code vs. at the parameter level
 - Alias
 - AllowNull
 - AllowEmptyString
 - AllowEmptyCollection
 - ValidateCount
 - ValidateLength
 - ValidatePattern
 - ValidateRange
 - ValidateScript
 - ValidateSet
 - ValidateNotNull
 - ValidateNotNullOrEmpty

[Alias()] Attribute



- Establishes an alternate name for a parameter
- An unlimited number of aliases can be assigned

Example:

MachineName and CN are aliases for the ComputerName Parameter

```
Param ([parameter()]) [alias("CN", "MachineName""Laptop", "Device")]  
[String[]] $ComputerName)
```

[ValidateSet()] Attribute

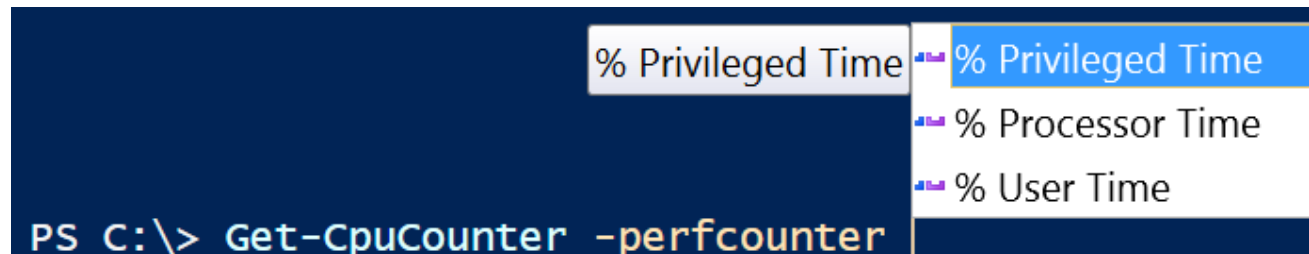


- Specifies a set of valid values for a parameter or variable
- PowerShell generates an error if a value does not exist in the set

Example:

Create a validate set to simplify returning CPU performance counter values

```
function Get-CpuCounter {  
    Param ([ValidateSet("% Processor Time", "% Privileged Time", "% User Time")] $perfcounter)  
    Get-Counter -Counter "\Processor(_Total)\$perfcounter"  
}
```



IntelliSense suggests allowed values

Null validation attributes



Task	Example
AllowNull: \$null is allowed	<pre>Param ([parameter(Mandatory=\$true)][AllowNull()][String] \$ComputerName)</pre>
AllowEmptyString: Empty string is allowed	<pre>Param ([parameter(Mandatory=\$true)][AllowEmptyString()][String]\$ComputerName)</pre>
AllowEmptyCollection: Empty collection is allowed	<pre>Param ([parameter(Mandatory=\$true)][AllowEmptyCollection ()][String[]]\$ComputerName)</pre>

Not null validation attributes



Task

Example

ValidateNotNull: \$null is not allowed

Param
([parameter(Mandatory=\$true)][validateNotNull()][String[]]\$ID)

ValidateNotNullOrEmpty: Neither \$null nor empty string allowed

Param
([parameter(Mandatory=\$true)][validateNotNullOrEmpty()][String[]]\$UserName)

Count validate attributes



Task	Example
ValidateCount: Specifies minimum and maximum accepted number of parameter values	<pre>Param ([ValidateCount(1, 5)][String[]] \$ComputerName)</pre>
ValidateLength: Specifies minimum and maximum accepted number of characters	<pre>Param ([ValidateLength(1, 15)][String[]] \$ComputerName)</pre>
ValidatePattern: Specifies a regular expression to compare to the parameter value	<pre>Param ([ValidatePattern("[0-9].[0-9].[0-9].[0-9]"))][String[]] \$ComputerName)</pre>

Range and script validate attributes



Task	Example
ValidateRange: Specifies a numeric range for each parameter value	<code>Param ([ValidateRange(0,10)] [Int] \$Attempts)</code>
ValidateScript: Specifies a "script" that is used to validate a parameter value PowerShell generates an error if the "script" returns "false" or if it throws an exception	<code>Param ([ValidateScript({\$_ -ge (get-date)})] [DateTime] \$EventDate)</code>

Demonstration

Parameter Validation



Questions?



Default Risk Mitigation Behavior

PowerShell uses combination of three mitigation mechanisms to determine whether confirmation prompts appear:

- `$ConfirmPreference`
 - Automatic variable controls confirmation prompts
 - Defaults to "High"
- `-Confirm` parameter can also override confirmation prompts
 - `-Confirm:$false` can be used to ignore all confirm prompts
- *ConfirmImpact* keyword defines the impact level of the function

[CmdletBinding()] Attribute - Risk Mitigation

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>,
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

Activates support for Risk Mitigation when set to \$True

–Whatif and –Confirm common parameter now present

[CmdletBinding()] Attribute - Risk Mitigation cont.

```
function <Name>
{
    [CmdletBinding(
        SupportsShouldProcess=<Boolean>
        ConfirmImpact=<String>,
        DefaultParameterSetName=<String>,
        HelpURI=<URI>,
        SupportsPaging=<Boolean>,
        PositionalBinding=<Boolean>)]
    Param ()
}
```

Values: "None", "Low", "Medium", "High"

Used to automatically invoke ShouldProcess depending on \$ConfirmPreference

Only use when SupportsShouldProcess is included

ConfirmImpact default is "Medium"

\$ConfirmPreference default is "High"

-Confirm and -WhatIf parameters override \$ConfirmPreference temporarily in scope of function.

Supporting Risk Mitigation in Functions

- Surround the code that makes changes to the system with an If() statement
- In the If() statement condition, the ShouldProcess() method enables PowerShell to determine whether the –WhatIf parameter has been used
- Multiple If() statements are allowed

```
If ($pscmdlet.ShouldProcess("Target", "Operation"))  
{  
    <Statement that makes changes>  
}
```

Supporting Risk Mitigation - Example

```
Function Kill-Process
{
    [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')]
    Param([String] $Name)

    $TargetProcess = Get-Process -Name $Name
    If ($pscmdlet.ShouldProcess($name, "Terminating Process"))
    {
        $TargetProcess.Kill()
    }
}
```



```
PS C:\> Kill-Process -Name notepad -WhatIf
What if: Performing the operation "Terminating Process" on target "notepad".
```

```
PS C:\> Kill-Process -Name notepad -Confirm
Confirm
Are you sure you want to perform this action?
Performing the operation "Terminating Process" on target "notepad".
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
(default is "Y"):
```

Confirmation Severity Levels

Default confirmations will be prompted when:

- ConfirmImpact value is greater than or equal to \$ConfirmPreference value

ConfirmImpact and \$ConfirmPreference Values	
None	No prompt(s)
Low	Prompt for confirmation before running cmdlets or functions with a low, medium, or high risk
Medium	Prompt for confirmation before running cmdlets or functions with a medium, or high risk
High	Prompt for confirmation before running cmdlets or functions with a high risk

\$ConfirmPreference
Default

Questions?



Function Comment-based Help

- Special help comment keywords can be used to write **Get-Help** topics
- **CTRL + J** (Start snippets) in ISE to get 'advanced function' with help topics

Syntax

```
# .< help keyword>  
# <help content>
```

-or -

```
<#  
    .< help keyword>  
    < help content>  
#>
```

Comment based keyboards

.SYNOPSIS

.DESCRIPTION

.PARAMETER <parameter name>

.EXAMPLE

.INPUTS

.OUTPUTS

.NOTES

.LINK

.COMPONENT

.ROLE

.FUNCTIONALITY

- Not case sensitive
- Capitalized for easier readability
- Don't need to use all of them if not needed
- Multiple **Example** keyword automatically numbers results in output view

Help example

- Functions should have **Get-Help** topics
- Very easy to add
- No excuse not to use in code

```
function Get-SysLogNN
{
    <#
        .SYNOPSIS
            Function that returns the most recent system event log entries.
        .DESCRIPTION
            Function that returns the most recent system event log entries.
            The number of items returned is determined by the parameter.
        .PARAMETER NumberOfEvents
        .EXAMPLE
            PS C:\> Get-SysLogNN -Log System -NumberOfEvents 10
        .EXAMPLE
            PS C:\> Get-SysLogNN -Log Application -NumberOfEvents 20
    #>
    Param ($log, $numberofevents)
    Get-EventLog -LogName $log -Newest $numberofevents
}
```

```
PS C:\> Get-Help Get-SysLogNN -full
```

NAME

Get-SysLogNN

SYNOPSIS

Function that returns the most recent system event log entries.

SYNTAX

Get-SysLogNN [[-log] <Object>] [[-numberofevents] <Object>]

[<CommonParameters>]

...

[OutputType()] Attribute

- Reports .NET object type that function returns
- Can combine with optional ParameterSetName parameter for different output types
- Use a null value when output is not a .NET type.

Output types

```
# multiple output types
```

```
[OutputType([<Type1>],[<Type2>],[<Type3>])]
```

```
# One or more parameter set names
```

```
[OutputType([<Type1>], ParameterSetName="<Set1>",<Set2>")]
```

```
[OutputType([<Type2>], ParameterSetName="<Set3>")]
```

[OutputType()] Attribute - Example

```
function Print-Hello {  
    [OutputType([double])]  
    Param ($Name)"Hello $Name"  
}
```

```
# Only used as a documentation note  
(Get-Command Print-Hello).OutputType  
# results
```

Name	Type
----	----

System.Double	System.Double
---------------	---------------

```
# Displays the actual .NET type returned by the command  
(Print-Hello -Name Johan).GetType().FullName  
# results
```

System.String

Questions?



Mastering parameters advanced functions

60 minutes

